

FR. CONCEICAO RODRIGUES COLLEGE OF ENGINEERING

Department of Computer Engineering Academic Year 2025-26

Rubrics for Lab Experiments

**Class : B.E. Computer
Engineering**

Subject Name :DC Lab

Semester : VIII

Subject Code : CSL801

Practical No:	4
Title:	Implement a message passing framework using middleware(Rabbitmq)
Date of Performance:	13/11/25
Roll No:	9913
Name of the Student:	Mark Lopes

Evaluation:

Performance Indicator	Below average	Average	Good	Excellent	Marks
On time Submission (2)	Not submitted (0)	Submitted after deadline (1)	Early or on time submission(2)	---	
Test cases and output (4)	Incorrect output (1)	The expected output is verified only a for few test cases (2)	The expected output is Verified for all test cases but is not presentable (3)	Expected output is obtained for all test cases. Presentable and easy to follow (4)	
Coding efficiency (2)	The code is not structured at all (0)	The code is structured but not efficient (1)	The code is Structured and efficient. (2)	-	
Knowledge(2)	Basic concepts not clear (0)	Understood the basic concepts (1)	Could explain the concept with suitable example (1.5)	Could relate the theory with real world application(2)	
Total					

Signature of the Teacher:

Aim: Implement a message-passing system using a middleware framework

Introduction:

A message queueing system is a software architecture pattern that enables communication between different parts of a distributed system by allowing them to exchange messages. In this system, messages are stored in a queue and are retrieved by consumers when they are ready to process them. This decouples the producers and consumers of messages, allowing them to operate independently and asynchronously.

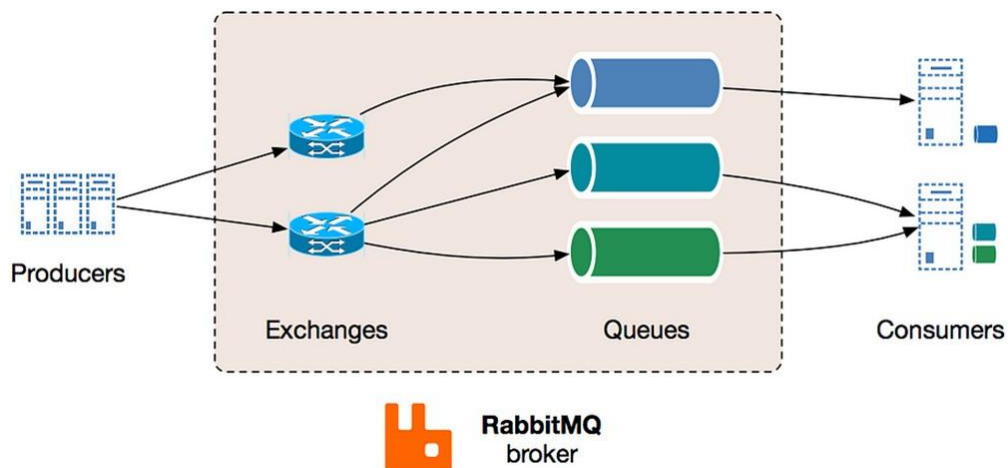
Message queueing systems can be implemented using various technologies, including open-source solutions like Apache Kafka, RabbitMQ, and ActiveMQ. In this practical we are going to implement a simple message queueing system using RabbitMQ.

RabbitMQ Model:

RabbitMQ is one of the most widely used message brokers, it acts as the message broker, “the mailman”, a microservice architecture needs.

RabbitMQ consists of:

1. producer — the client that creates a message
2. consumer — receives a message
3. queue — stores messages
4. exchange — enables to route messages and send them to queues



The system functions in the following way:

1. producer creates a message and sends it to an exchange
2. exchange receives a message and routes it to queues subscribed to it
3. consumer receives messages from those queues he/she is subscribed to

Implementation:

We are going to implement a job manager as described in the below figure.

Components of our message queueing system are:

- Publisher – produces jobs/messages into the queue
- Consumers – consumes the jobs
- RabbitMQ broker – contains the exchange and queue
- Connections – denoted by double-sided arrows
- Channels – denoted by colourful bands within the connections

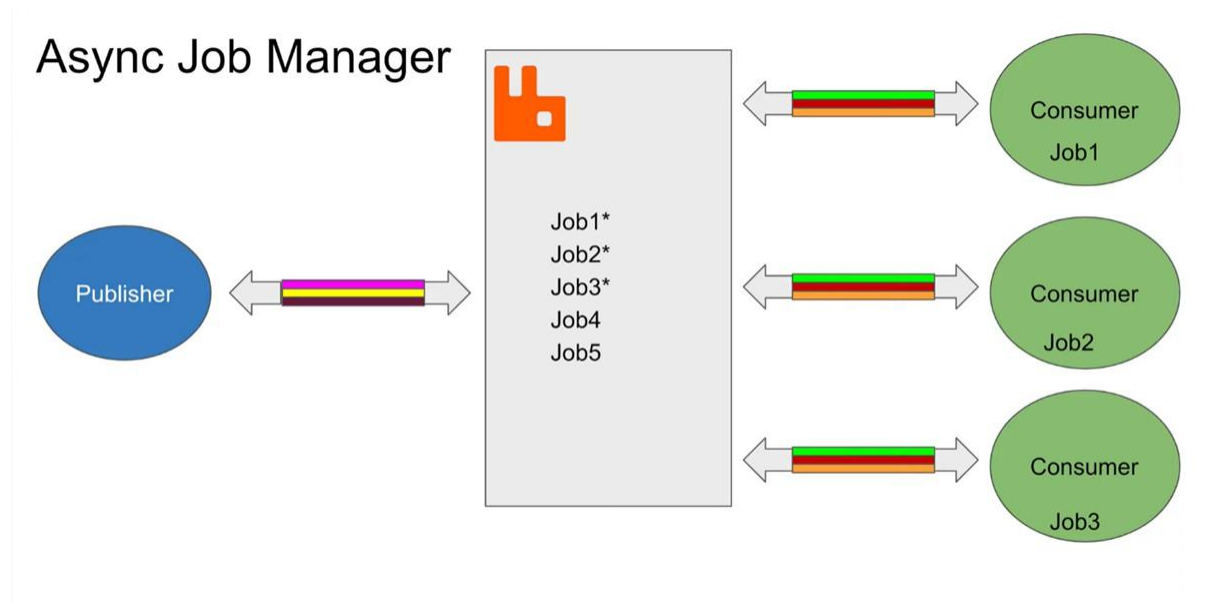


Fig. Message Queuing System

Technologies Used:

- Docker
- RabbitMQ Image
- Node.js
- amqplib Library

Step 1: Run RabbitMQ's Docker Image

Step 2: Write a Producer Program

Step 3: Write a Consumer Program

Publisher.js:

```
const amqp = require('amqplib');

const RABBITMQ_URL = 'amqp://localhost'; // Use 'amqp://localhost' if running on host or
appropriate Docker network setting

const QUEUE_NAME = 'job_queue';

async function connectAndPublish() {
  try {
    // 1. Connect to RabbitMQ server

    const connection = await amqp.connect(RABBITMQ_URL);
    const channel = await connection.createChannel();

    // 2. Assert the queue (makes sure the queue exists, or creates it)
    // 'durable: true' ensures the queue survives a broker restart
    await channel.assertQueue(QUEUE_NAME, {
      durable: true
    });

    // The messages (jobs) to be sent
    const jobs = [
      "Job 1: Process User Data",
      "Job 2: Generate Report PDF",
      "Job 3: Send Email Notification",
      "Job 4: Backup Database"
    ];
```

```

for (let i = 0; i < jobs.length; i++) {

  const msg = jobs[i];

  // 3. Send the message to the queue

  // `Buffer.from(msg)` converts the string to a Buffer

  // `persistent: true` marks the message as persistent, so it survives a broker restart

  channel.sendToQueue(QueueName, Buffer.from(msg), {

    persistent: true

  });

  console.log(`✅ [Producer] Sent job: "${msg}"`);

}

// Close the connection after a short delay to ensure messages are sent

setTimeout(() => {

  connection.close();

  process.exit(0);

}, 500);

} catch (error) {

  console.error("❌ Error in Producer:", error.message);

  process.exit(1);

}

}

connectAndPublish();

```

Consumer.js

```
const amqp = require('amqplib');

const RABBITMQ_URL = 'amqp://localhost'; // Use 'amqp://localhost' or appropriate Docker
network setting

const QUEUE_NAME = 'job_queue';

async function connectAndConsume() {
  try {
    // 1. Connect to RabbitMQ server

    const connection = await amqp.connect(RABBITMQ_URL);
    const channel = await connection.createChannel();

    // 2. Assert the queue (must be the same as the producer's)
    await channel.assertQueue(QUEUE_NAME, {
      durable: true
    });

    // Ensure only one message is delivered to the consumer at a time
    // This distributes jobs more evenly across multiple consumers (if running)
    channel.prefetch(1);

    console.log("🕒 [Consumer] Waiting for jobs in %s. To exit press CTRL+C",
    QUEUE_NAME);

    // 3. Start consuming messages
    channel.consume(QUEUE_NAME, (msg) => {
```

```

    if (msg !== null) {

        const jobContent = msg.content.toString();

        console.log(` ➡ [Consumer] Received job: '${jobContent}'`);

        // Simulate processing time

        const processingTime = Math.floor(Math.random() * 3000) + 1000; // 1 to 4
seconds

        console.log(` [Consumer] Processing job... (simulated time:
${processingTime}ms)`);

        setTimeout(() => {

            // 4. Acknowledge the message (tells RabbitMQ the job is done)

            channel.ack(msg);

            console.log(` ✅ [Consumer] Finished and acknowledged: '${jobContent}'`);

        }, processingTime);

    }

}, {

    // `noAck: false` is critical for reliable messaging. The consumer must explicitly
    acknowledge.

    noAck: false

});

} catch (error) {

    console.error(" ❌ Error in Consumer:", error.message);

    process.exit(1);

}

}

connectAndConsume();

```



```
welcome to hallownest, little ghost...

Hallownest: C:\Users\Mark Lopes\Desktop\college\Sem_8\DC\exp4\rabbit-lab
> node publisher.js
>>
[✓] [Producer] Sent job: 'Job 1: Process User Data'
[✓] [Producer] Sent job: 'Job 2: Generate Report PDF'
[✓] [Producer] Sent job: 'Job 3: Send Email Notification'
[✓] [Producer] Sent job: 'Job 4: Backup Database'

Hallownest: C:\Users\Mark Lopes\Desktop\college\Sem_8\DC\exp4\rabbit-lab
> node consumer.js
>>
[⌚] [Consumer] Waiting for jobs in job_queue. To exit press CTRL+C
[➡] [Consumer] Received job: 'Job 1: Process User Data'
[Consumer] Processing job... (simulated time: 1907ms)
[✓] [Consumer] Finished and acknowledged: 'Job 1: Process User Data'
[➡] [Consumer] Received job: 'Job 2: Generate Report PDF'
[Consumer] Processing job... (simulated time: 1337ms)
[✓] [Consumer] Finished and acknowledged: 'Job 2: Generate Report PDF'
[➡] [Consumer] Received job: 'Job 3: Send Email Notification'
[Consumer] Processing job... (simulated time: 1624ms)
[✓] [Consumer] Finished and acknowledged: 'Job 3: Send Email Notification'
[➡] [Consumer] Received job: 'Job 4: Backup Database'
[Consumer] Processing job... (simulated time: 1266ms)
[✓] [Consumer] Finished and acknowledged: 'Job 4: Backup Database'
```



RabbitMQ™

RabbitMQ 3.13.7 Erlang 26.2.5.16

Refreshed 2025-11-24 20:58:58

Refresh every

Virtu

Cluster rabbit

User gu

Overview

Connections

Channels

Exchanges

Queues and Streams

Admin

Queue job_queue

Overview

Queued messages last ten minutes ?



Ready

Unacked

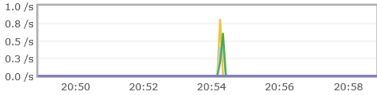
Total

0

0

0

Message rates last ten minutes ?



Publish

Deliver (manual ack)

Deliver (auto ack)

Consumer ack

Redelivered

Get (auto ack)

Get (empty)

0.00/s

0.00/s

0.00/s

0.00/s

0.00/s

0.00/s

Details

Features

Policy

Operator policy

Effective policy definition

State

Consumers

Consumer capacity

idle

1

94%

Messages ?

Message body bytes ?

Process memory ?

0

0 B

10 KiB

Total

Ready

Unacked

In memory

Persistent

Transient, Paged Out

0

0

0

0

0

0

queue storage version: 1

Consumers (1)

Channel ?	Consumer tag	Ack required	Exclusive	Prefetch count	Active ?	Activity status	Consumer Timeout	Arguments
172.17.0.1:34178 (1)	amq.ctag-5zq_41O62alNLWco4ceQLg	•	○	1	•	up	1800000	

Bindings (1)

From	Routing key	Arguments
(Default exchange binding)		

⇕

This queue

Add binding to this queue

Conclusion:

This lab successfully implemented a basic, yet robust, message-passing system using Node.js and the RabbitMQ broker. By utilizing key RabbitMQ concepts—the Queue, Publisher (Producer), and Consumer—we achieved Decoupling, Asynchrony and persistence.

Postlab:

1) What is message queuing?

Message queuing is a software pattern that enables asynchronous communication between different parts of a distributed system. It allows a producer to send a message to a queue and immediately continue its work without waiting for the consumer to respond. This helps in decoupling the services and ensures smooth, independent operation of different system components.

2) What are the benefits of message queuing?

Message queuing makes producers and consumers independent, so even if one service fails, the other continues to work without being affected. If the consumer service is unavailable, the message is not lost and simply waits in the queue until it can be processed. It also helps in handling traffic spikes by queueing extra messages and preventing consumer services from becoming overloaded.

